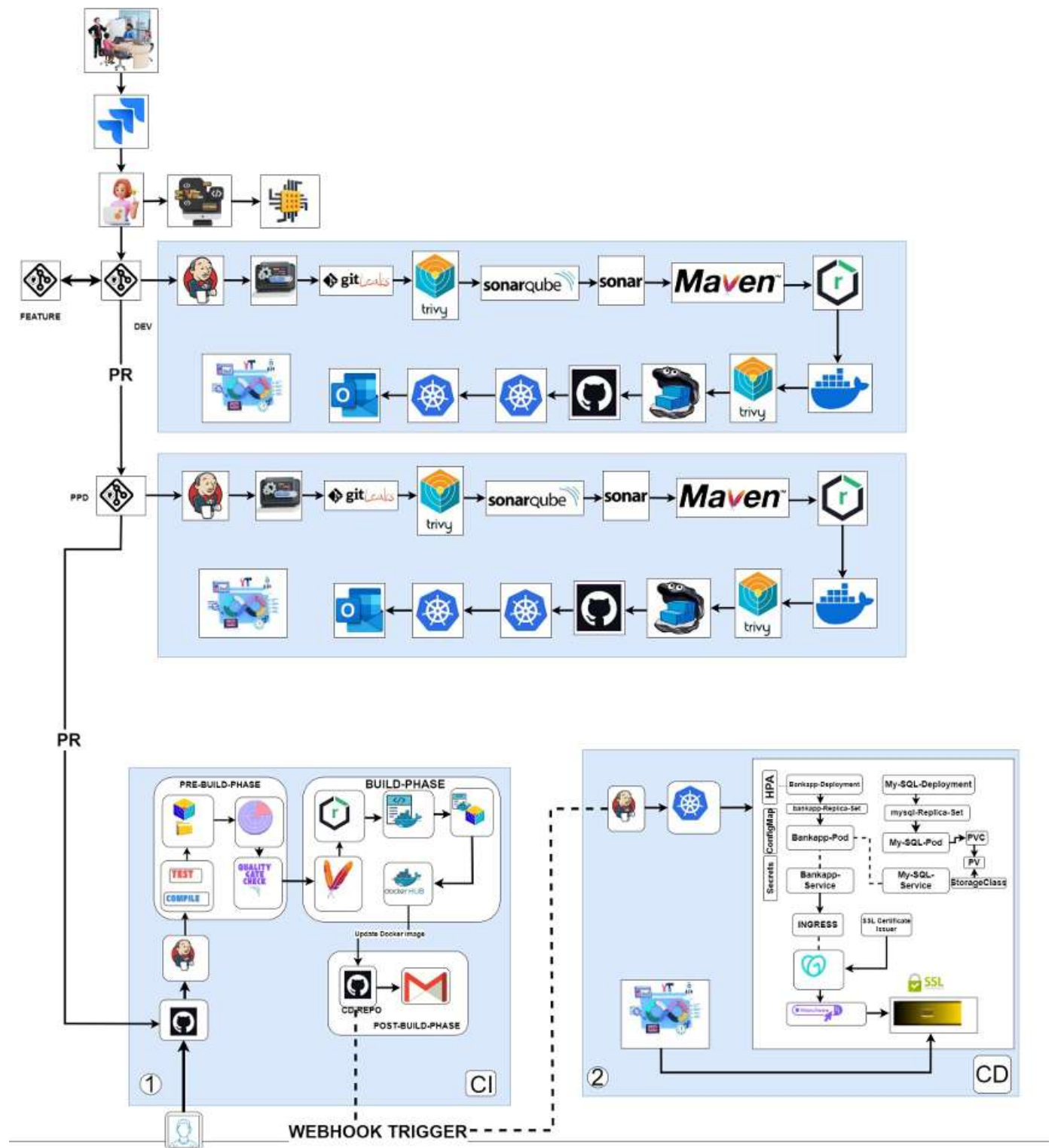


5. Jenkins based CI/CD pipeline

Wednesday, July 15, 2026 5:12 PM



1. Project Planning & Development Kickoff

The pipeline begins even before any code is written. It starts from **project management and task planning**, where business requirements are broken down into small deliverables using tools like **JIRA**.

Tools:

- **JIRA** – Sprint planning, task tracking, and linking code to features
- **Git (GitHub/GitLab/Bitbucket)** – Source code repository with branching strategy

Flow:

- A new **user story** is created in JIRA:
"As a user, I want to login securely with email and password."
 - The story is broken down into:
 - Create login API
 - Add validation
 - Connect to DB
 - Write unit tests
 - A developer is assigned the task and begins coding.
-

2. Local Development & Feature Branching

The developer pulls the latest code and creates a **feature branch** using Git. This ensures isolated development without affecting the main codebase.

Example:

```
git checkout -b feature/login-api
```

Tools:

- **VS Code, IntelliJ IDEA** – Code editors
- **Docker Compose / Minikube** – Run the full app stack locally
- **Postman** – API testing
- **Prettier/ESLint** – Code formatting

During development:

- Business logic is implemented.
- Environment variables are defined.
- Unit tests are written.
- `.env` or `config.yml` is prepared.

3. Git Push & Pull Request Lifecycle

After completing development, the code is pushed to the remote repository and a **Pull Request (PR)** is raised from the feature branch to dev.

Example:

```
git push origin feature/login-api
```

A webhook is triggered to Jenkins when the PR is raised, which kicks off the **automated CI pipeline**.

4. Jenkins CI Pipeline Triggered

The Jenkins master receives the webhook and triggers the pipeline defined in the Jenkinsfile. Jenkins checks out the code, sets up the environment, and runs all configured stages.

Pipeline Configuration:

```
pipeline {
  agent any
  stages {
    stage('Pre-Build') { ... }
    stage('Build') { ... }
    stage('Test') { ... }
    stage('Quality Gate') { ... }
    stage('Security Scan') { ... }
    stage('Docker Build & Push') { ... }
    stage('Deploy to Dev') { ... }
  }
}
```

5. Pre-Build Phase

This phase validates the code structure and checks for early errors.

Tasks:

- Code formatting checks (Prettier, Black)
- Linting (ESLint, Checkstyle)
- Secrets scanning (Gitleaks)
- Dependency resolution (npm install, mvn clean install)

Example:

If Gitleaks detects a hardcoded secret (like an AWS access key), it halts the pipeline before any further action, preventing exposure.

6. Build Phase

The code is compiled and packaged.

Tool: Maven

- Clean build: mvn clean install
- Creates .jar or .war file in /target

Jenkins Output:

[INFO] BUILD SUCCESS

[INFO] Final artifact: target/myapp-1.0.0-SNAPSHOT.jar

7. Test Phase

All unit tests are executed and test reports are generated.

Tools:

- **JUnit, TestNG** – Unit test framework
- **Jacoco** – Code coverage
- **Jenkins JUnit plugin** – Displays results in dashboard

Example Output:

Total Tests: 45 | Passed: 43 | Failed: 2 | Coverage: 88%

If test coverage falls below 80%, Jenkins can be configured to fail the pipeline.

8. Code Quality Analysis (SonarQube)

Static analysis is conducted using **SonarQube**. Jenkins integrates with a running SonarQube server to analyze the code.

Metrics:

Bugs
Code Smells
Vulnerabilities
Duplications
Maintainability Index

Quality Gate:

- Fails build if:
 - Coverage < 80%
 - Any new critical issue

Jenkins Console:

SonarQube Quality Gate: FAILED

Reason: 3 Critical Bugs, 2 Vulnerabilities

9. Security Scanning (Trivy)

The .jar or Docker image is scanned for CVEs using **Trivy**.

Example Output:

```
trivy fs ./target/myapp.jar
```

- Finds issues in dependencies (e.g., log4j, spring-core)
 - Can break pipeline if severity = HIGH or CRITICAL
-

10. Docker Image Build & Push

Jenkins builds a Docker image of the app and pushes it to a container registry.

Dockerfile:

```
FROM openjdk:17
```

```
COPY target/myapp.jar app.jar
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Commands:

```
docker build -t registry.com/myapp:dev .
```

```
docker push registry.com/myapp:dev
```

Output:

Successfully pushed myapp:dev to Docker Hub

11. Notifications

The build result is communicated through various channels:

Examples:

- **Slack:** "Build #105 Passed 
- **Email:** "Subject: Jenkins Pipeline Failed 
- **GitHub Status Check:** PRs show CI status inline

12. Deploy to Kubernetes (Dev)

Jenkins connects to the **Kubernetes DEV cluster** and applies manifests.

Process:

- Set KUBECONFIG in Jenkins agent
- Use kubectl apply or Helm chart

YAML Sample:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  template:
    spec:
      containers:
        - image: registry.com/myapp:dev
```

- DEV environment has mock services or sandbox data
- Monitoring enabled (Prometheus/Grafana)

13. Merge to QA & PPD

Once verified in DEV:

- PR from dev → QA
- PR from QA → ppd
- Jenkins runs same CI pipeline
- Deploys to PPD cluster with real integrations

PPD Key Characteristics:

- Real secrets fetched from **HashiCorp Vault**
- Feature flags turned ON
- Load testing / staging users perform UAT

14. Merge to Main & PROD Deployment

The final merge triggers the **PROD release pipeline**.

Features:

- Blue/Green or Canary deployment via Helm
- TLS with **cert-manager**

- Live traffic routing via **Ingress**

Helm Command:

```
helm upgrade --install myapp ./charts/myapp --values prod-values.yaml
```

 15. Kubernetes Resources Breakdown

Resource	Role
Deployment	Rollouts, rolling updates
ReplicaSet	Maintains desired pod count
Pod	Hosts containers
Service	Exposes pods inside cluster
Ingress	Routes traffic to services
PVC	Persistent storage for DB or state
Secret/ConfigMap	Injects runtime config and credentials

16. Observability and Post-Deploy

- **Prometheus:** Monitors metrics (CPU, memory, HTTP errors)
 - **Grafana:** Dashboards for health and business KPIs
 - **Loki/ELK:** Centralized logs
 - **Sentry:** Real-time error tracking
-

17. Security Best Practices

- Enforce non-root users in Docker
- Regular base image scans
- Vault-injected secrets (never hardcoded)
- RBAC for Jenkins and cluster access
- Image Signing with Cosign (optional)